OlomiPay

Partner Onboarding & Compliance Pack

Confidential — for partner/regulator review only

Generated 2026-06-08

00 — Company & KYB Summary

> KYB (Know-Your-Business) basics a partner collects up front. Fill the bracketed fields — they are legal facts only you can supply.

1. Legal entity

Field	Value
Registered company name	[ ]
Trading name / brand	OlomiPay
Country of incorporation	[ ] (e.g. Tanzania)
Company registration number	[ ]
TIN / tax registration	[ ]
Registered address	[ ]
Operating address	[ ]
Date of incorporation	[ ]
Website	[ ] (e.g. https://olomipay.app)

2. Ownership & control

Field	Value
Directors	[Full names, IDs]
Ultimate Beneficial Owners (>25%)	[Full names, % holding, IDs]
Group structure / parent	[None / details]

3. Key people

Role	Name	Contact
CEO / Founder	[ ]	[ ]
CTO / Technical lead	[ ]	[ ]
Compliance / MLRO (Money Laundering Reporting Officer)	[ ]	[ ]

Finance / Treasury	[ ]	[ ]

4. Licensing & regulatory posture

- OlomiPay is **not** independently licensed as a PSP/EMI at this time.
- OlomiPay intends to operate the local fiat (TZS / M-Pesa) leg **under the licensed infrastructure of its on/off-ramp partner**, and is seeking written confirmation that its use case is covered.
- OlomiPay operates a **custodial** USD-denominated wallet (see Doc 04) and applies AML/KYC controls accordingly (see Doc 05).
- Local legal counsel: [firm / status].

5. Banking & settlement

Field	Value
Corporate bank(s)	[ ]
Stablecoin used	USDC (Stellar network)
USD treasury rails	[e.g. Circle Mint — status]
Fiat on/off-ramp partner	[e.g. Yellow Card]

6. Supporting documents to attach

- [ ] Certificate of incorporation
- [ ] Memorandum & Articles / company constitution
- [ ] Director & UBO IDs + proof of address
- [ ] Proof of company address (utility/lease)
- [ ] AML/KYC policy (Doc 05 + signed policy)
- [ ] Org chart
- [ ] Source-of-funds / business plan summary (Doc 01)

01 — Business Overview

1. What OlomiPay is

OlomiPay is a **payments-first financial app** for Africa (initial market: Tanzania). It presents users with a single **US-dollar wallet** and lets them:

- **Add money** from mobile money (M-Pesa) — local currency is converted to a USD-denominated balance.
- **Send money** instantly to other users by phone number or QR.
- **Chat** with built-in payments (send/request money inside a conversation).
- **Withdraw** back to mobile money (and, where enabled, bank).
- **Save & grow** — interest-bearing savings, goals, and group savings (chama).
- **Send across borders** and **cash in/out via local agents** (where enabled).

The underlying settlement asset is the **USDC stablecoin on the Stellar network**, but this is **invisible to end users** — they see only USD. OlomiPay is a wallet/payments product, **not** a crypto exchange or trading venue.

2. Target users & market

- **Primary:** everyday consumers and small businesses in Tanzania who already use mobile money, want a stable USD store of value (inflation hedge), and need cheap, instant payments + chat.
- **Geography:** Tanzania first; designed to extend to other African/Asian markets via the partner's licensed corridors.

3. Products & use cases

Product	Description
USD wallet	Hold value in USD; balance is USDC held on Stellar (hidden).
Add money	M-Pesa → USD balance, via the licensed on/off-ramp partner.
Send / Request	P2P transfer by phone/QR; in-chat payments & requests.
Withdraw	USD balance → M-Pesa (and bank, where enabled).
Savings / Goals	Interest-bearing savings + goal pots; platform-managed vault.
Group savings (Chama)	Rotating savings groups.
Cross-border send	Payout to recipients in other supported currencies.

Cash agents	Human agents for cash-in/cash-out (where enabled).
Bills / Merchant / Payroll	Bill pay, merchant collection, bulk payouts.

4. Revenue model

- A small **platform fee** (≈1%) on outbound transfers/conversions.
- FX/spread on conversions is handled by the on/off-ramp partner.
- Future: yield products, business tools, premium features.

5. What OlomiPay is NOT

- Not a crypto exchange, trading platform, or speculative venue.
- Not a lender of record for third-party credit (any lending is P2P/feature-gated).
- Not independently licensed as a PSP/EMI — it operates the fiat leg under a licensed partner's infrastructure (see Docs 00 & 07).

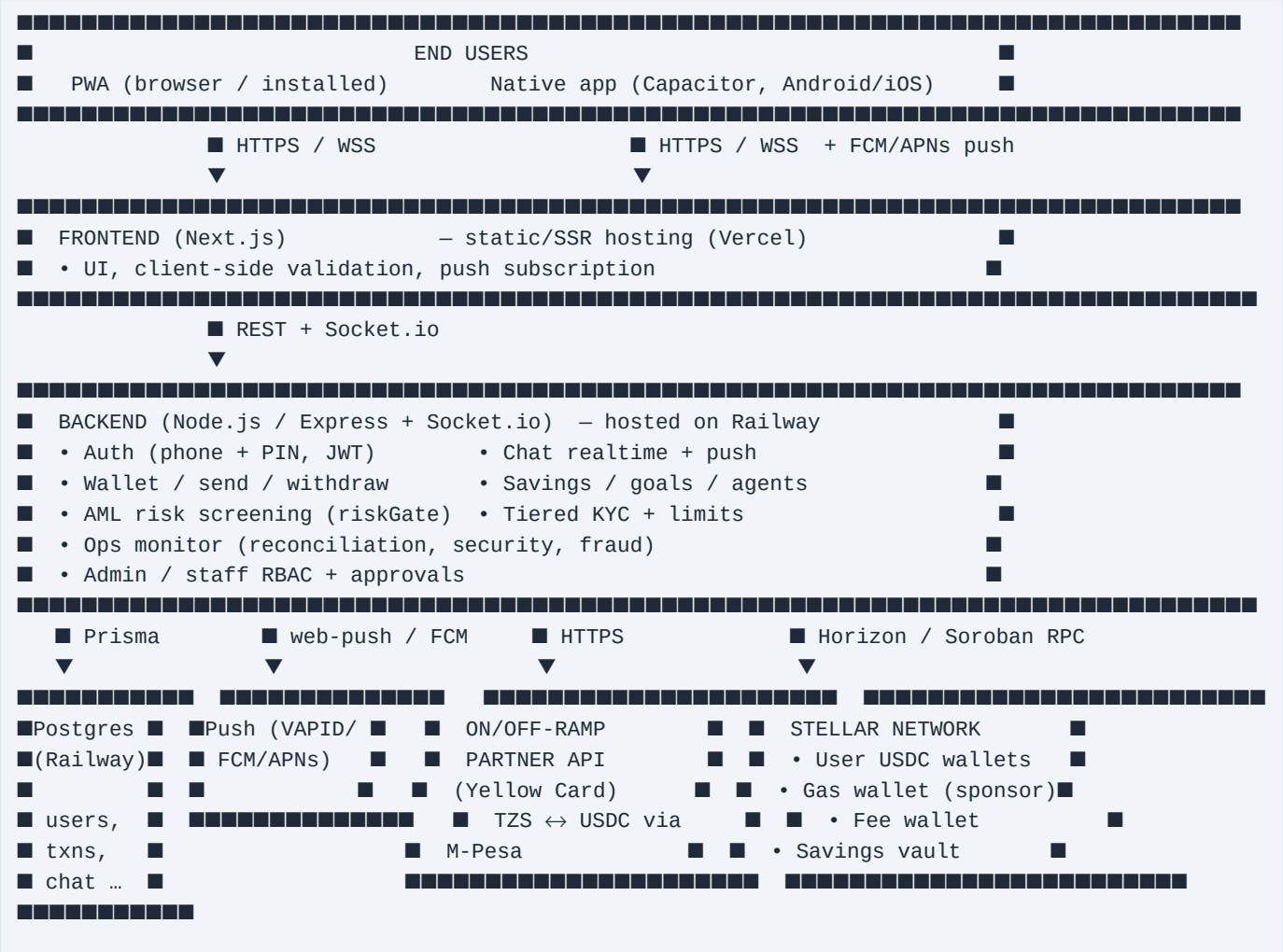
6. Scale & status

- Stage: [pre-launch / pilot / live — fill in].
- Expected volumes: [monthly transaction count & value estimate].
- Average transaction size: [ ].
- These figures are provided to the partner in the pre-integration form.

02 — Technical Architecture

1. High-level overview

OlomiPay is a three-tier system: a client (web PWA + native mobile shell), a backend API + realtime service, and a settlement layer on the Stellar network. The local fiat (M-Pesa) leg is handled entirely by the **licensed on/off-ramp partner**; OlomiPay never directly touches the mobile-money rails.



2. Components & technology

Layer	Technology	Hosting
Web frontend	Next.js 14 (App Router), React, TypeScript	Vercel
Native apps	Capacitor (Android/iOS) loading the live web app	App stores / APK
Backend API	Node.js, Express, TypeScript	Railway

Realtime	Socket.io (WebSocket + long-polling, optional Redis adapter)	Railway
Database	PostgreSQL via Prisma ORM	Railway (managed)
Settlement	Stellar (USDC asset); Soroban for token transfers	Public Stellar network
Push	Web Push (VAPID) + FCM (Android) / APNs (iOS)	Google/Apple
Error/uptime	Sentry; internal ops monitor + alerting	—

3. Settlement layer (Stellar)

- Users hold **USDC** in individual Stellar accounts.
- A **gas wallet** sponsors account reserves and pays network fees via fee-bump transactions, so users never need to hold the native asset (XLM).
- A **dedicated fee wallet** (separate from the gas wallet) collects the platform fee with its own USDC trustline.
- A **savings vault** address holds funds placed into savings/goals.
- Outbound user→user transfers are signed with the user's key (PIN-authorised); platform-originated movements (e.g. savings withdrawal payout) are signed by platform-controlled wallets. See Doc 04 for the key model.

4. Authentication & sessions

- **End users:** phone number + 6-digit PIN → JWT access token (+ long-lived refresh token). PINs are stored only as **bcrypt** hashes.
- **Staff/admin:** separate username + password identities with **role-based access control** (super-admin, department heads, department staff) and a **multi-approver workflow** for sensitive actions.
- Account lockout after repeated failed attempts (see Doc 05/06).

5. Integrations

Integration	Purpose	Direction
On/off-ramp partner (Yellow Card)	TZS ↔ USDC via M-Pesa (deposit/withdraw/remit)	OlomiPay → partner API + partner webhooks
Circle (optional/treasury)	USD ↔ USDC treasury; bank payouts	OlomiPay → Circle API
Stellar Horizon / Soroban RPC	On-chain balances, transfers	OlomiPay → network

FCM / APNs / Web Push	Notifications	OlomiPay → device

6. Environments & deployment

- Source control: Git (GitHub). CI/CD via Vercel (frontend) and Railway (backend).
- Secrets are held as platform environment variables (see Doc 06 for handling).
- Separate testnet/sandbox configuration is supported for partner sandbox testing.

7. Data stored (high level)

- User: phone, name, KYC level/details, encrypted wallet key, Stellar public key.
- Transactions: type, amount (USDC), status, counterparties, timestamps, refs.
- Chat: conversations, messages (see Doc 06 re content handling), receipts.
- Compliance: security events, risk reviews, admin audit log, approvals.
- Full schema available to the partner on request under NDA.

03 — Fund Flows

This document traces money for each operation: where value enters, where it sits, who signs, and where it exits. The **local fiat (TZS/M-Pesa) leg is always executed by the licensed on/off-ramp partner**; OlomiPay holds value as USDC on Stellar.

Legend: **User wallet** = the user's Stellar USDC account · **Platform wallets** = gas / fee / savings-vault / treasury accounts controlled by OlomiPay · **Partner** = licensed on/off-ramp (Yellow Card).

---

1. Add money (deposit) — TZS → USD balance

```
User (M-Pesa, TZS)
  ■  1. user requests deposit of X TZS in app
  ▼
Partner (Yellow Card)  — collects TZS via M-Pesa, converts to USDC
  ■  2. partner pays out USDC (net of partner fee)
  ▼
User wallet (USDC on Stellar)   — balance shown to user as USD
```

- OlomiPay never touches TZS; the partner performs collection + conversion under its licence and confirms via webhook.
- Idempotency: each deposit has an internal reference; the partner order is keyed to it.

2. Send money (P2P) — user → user

```
Sender wallet (USDC)
  ■  1. sender authorises with PIN (signs transfer)
  ■  2. AML screen (riskGate) + tier-limit check
  ▼
Platform fee wallet  ← ~1% platform fee
  +
Recipient wallet (USDC)  ← net amount
```

- On-chain transfer on Stellar; gas paid by the platform gas wallet (fee-bump), so the user needs no native asset.
- Recipient sees an instant USD credit + notification.

3. Withdraw — USD balance → TZS (M-Pesa)

```
User wallet (USDC)
  ■  1. user authorises with PIN
  ■  2. tier-limit + AML checks
  ▼
Platform (debits user's USDC)
  ■  3. instruct partner to pay out
  ▼
Partner (Yellow Card)  — converts USDC → TZS, pays user's M-Pesa
  ▼
```

```
User (M-Pesa, TZS)
```

4. Cross-border send (remittance)

```
Sender wallet (USDC)  → platform debits  → Partner converts USDC → local currency
                                         → recipient's mobile money (other country)
```

- Same control set as withdrawal (PIN, tier limits incl. a remittance feature gate, AML screen). Real payout requires the partner's production corridors.

5. Savings / Goals

```
User wallet (USDC)
  ■  1. user authorises deposit with PIN
  ▼
Savings vault (platform-controlled wallet)   — accrues interest
  ■  2. on withdrawal, platform signs payout
  ▼
User wallet (USDC)
```

- While saved, funds sit in a **platform-controlled vault** → this portion is explicitly **custodial** (see Doc 04). Withdrawals are platform-signed.

6. Cash agents (cash-in / cash-out)

```
Cash-in  : customer gives agent physical cash → agent wallet → customer wallet (USDC)
Cash-out : customer authorises (PIN) → customer wallet → agent wallet → agent gives cash
```

- Agents are KYC-verified, limit-capped, and monitored for velocity/abandonment (see Doc 05). Cash-out uses short-lived confirmation codes.

7. Fee handling & treasury

- The ~1% platform fee accrues in a **dedicated fee wallet** (separate from gas).
- Fees can be swept to top up the gas wallet (operational sustainability).
- Platform reserves vs. user liabilities are **reconciled automatically** every cycle (alerts if platform USDC < user liabilities). See Doc 05/06.

8. Settlement & reconciliation summary

Operation	Fiat leg (partner)	On-chain leg (OlomiPay)	Signer
Deposit	TZS→USDC	USDC credited to user	Partner
Send	—	USDC user→user (+fee)	User (PIN)
Withdraw	USDC→TZS	USDC debited from user	User (PIN) + platform

Remittance	USDC→local	USDC debited from user	User (PIN) + platform
Savings in/out	—	user↔vault	User (PIN) in; platform out
Agent cash-in/out	—	agent↔user	Agent / user (PIN)

04 — Custody & Key Management

> This document states OlomiPay's custody position **accurately**. Overstating "non-custodial" to a partner or regulator is a misrepresentation; the design below is custodial and is described as such.

1. Custody position (plain statement)

OlomiPay operates a CUSTODIAL USD-denominated wallet. The operator has the technical ability to control user funds. OlomiPay is therefore responsible for safeguarding client assets and applies AML/KYC and security controls to that standard (Docs 05 & 06).

2. How user wallets are created

Each user has an individual Stellar account holding USDC. The keypair is **deterministically derived** from a single server-side master secret and the user's phone number:

```
seed     = HMAC-SHA256(WALLET_DERIVATION_SECRET, "olomipay-wallet-v1:" + phone)
keypair  = ed25519(seed)
```

Implications, stated plainly: - The operator, holding `WALLET_DERIVATION_SECRET`, can **re-derive any user's private key from their phone number** — no user PIN required. This is what makes the model **custodial**, and it powers **phone-based account recovery** (a user who reinstalls or changes device regains the same wallet + funds). - The user's stored secret is **also** kept encrypted at rest (defence in depth, below), but the operator does not depend on that copy.

3. Encryption at rest (defence in depth)

The user's secret key is additionally stored encrypted with **AES-256-GCM**, where the key is derived as:

```
encKey = PBKDF2(userPIN, salt = SHA256(phone + ENCRYPTION_KEY), 310,000, SHA-512)
```

- This protects the stored copy against an attacker who steals the **database but not** the server secrets.
- PINs themselves are never stored — only **bcrypt** hashes (cost 12).
- It does **not** make the system non-custodial (see §2).

4. Platform-controlled wallets

Wallet	Purpose	Control
Gas wallet	Sponsors account reserves + pays network fees (fee-bump)	Platform
Fee wallet	Collects the ~1% platform fee (own USDC trustline)	Platform

Savings vault	Holds funds placed into savings/goals	Platform
Treasury (as configured)	Settlement / payouts	Platform

Savings/goal balances **sit in a platform-controlled wallet** and payouts are platform-signed — explicitly custodial for that portion.

5. Transaction signing

- **User-initiated transfers** (send, withdraw, savings deposit, agent cash-out) require the user's **PIN**, which authorises decryption/derivation and signing.
- **Platform-initiated payouts** (savings/goal withdrawals, fee sweeps, partner settlement) are signed by platform-controlled wallets.
- Server-side **risk screening** and **tier-limit checks** run before signing.

6. Reserves & solvency

- User balances are backed 1:1 by USDC held on Stellar.
- An automated **reconciliation** check compares platform USDC holdings against aggregate user liabilities every monitoring cycle and **alerts** if holdings fall below liabilities (see Doc 05/06).

7. Key risk concentration & safeguards

`WALLET_DERIVATION_SECRET` (and `ENCRYPTION_KEY`) are the highest-value secrets: compromise of the derivation secret would expose **all** user funds. Controls:

Control	Status
Secrets stored as environment variables, access-restricted	In place
Separation of gas vs. fee wallets	In place
Rotation support for the at-rest encryption key (`ENCRYPTION_KEY_PREVIOUS`)	In place
Move master key/signing into a dedicated **KMS/HSM-backed signer**	Planned / recommended
Least-privilege access to production secrets; audit of access	[state current status]
Periodic security review / penetration test	[planned / done — fill in]

> **Roadmap note:** isolating signing into a KMS/HSM service (so application servers never see raw user keys) is the top security hardening item and is recommended before scaling balances.

8. Summary for the reviewer

- Model: **custodial** wallet, USD value held as USDC on Stellar.
- Operator control: yes (deterministic derivation + platform vaults).
- User authorisation: PIN-gated for user-initiated movements.
- Safeguarding: 1:1 USDC backing + automated reconciliation + at-rest encryption.
- Primary residual risk: master-secret concentration — mitigation roadmap above.

05 — AML / KYC & Compliance Controls

> Describes the controls implemented in the platform plus the policy wrapper. Items marked **[policy]** are organisational processes; **[in progress]** marks controls planned but not yet fully automated. Be accurate when submitting — do not present planned controls as live.

1. Customer Due Diligence (KYC) — tiered model

Access and limits scale with the level of identity verification. Limits are enforced **server-side** on every outbound action (fail-closed on breach).

Level	Verification	Per-tx	Daily	Unlocks
0 New	Phone number only	$50	$100	Send (small)
1 Basic	+ full name	$300	$1,000	+ cross-border send
2 Verified	+ government ID (NIDA / passport / voter / licence)	$3,000	$10,000	+ cash agents, bank withdrawal
3 Enhanced	Admin-granted (business / high-volume)	$25,000	$100,000	Highest

- ID details captured at Level 2: ID type, ID number, legal name.
- A monthly limit also applies per level; usage is measured against rolling 24h + 30d **confirmed + pending** outflows so in-flight transactions cannot bypass caps.
- **ID document verification provider** integration (e.g. Smile Identity): **[in progress]** — production approvals must be gated on a real identity provider rather than auto-approval.

2. Transaction monitoring (automated)

A pre-flight risk screen runs before any value-moving transaction: - **Hard cap:** single transfers above an absolute ceiling are **blocked**. - **Review thresholds:** large amounts, high velocity (per-minute / per-hour), and new-recipient + large-amount patterns are **scored and flagged** for review (logged to a risk-review store). - **Frozen accounts** are blocked outright. - Design is fail-open on internal error (a screening bug must not block all payments) **except** the hard cap and frozen-account checks, which fail-closed.

3. Ongoing / behavioural monitoring (ops monitor, ~20-min cycle)

- **Solvency/reconciliation:** alerts if platform USDC < aggregate user liabilities.
- **Security signals:** mass account lockouts (possible credential stuffing), IPs with high failed-login counts.
- **Agent network:** low float, abnormal transaction velocity, and high failed/abandoned cash-out ratios.

- Alerts are delivered to operators (webhook + super-admin push), throttled.

4. Sanctions / PEP screening

- **[in progress / policy]** — name screening against sanctions and PEP lists at onboarding and on an ongoing basis. Where the on/off-ramp partner performs screening on the fiat leg, OlomiPay relies on and supplements it. State the current provider/process here: [ ].

5. Record keeping

- All transactions are persisted with type, amount, status, counterparties, timestamps, and references (USDC + on-chain tx ids).
- Security events, risk reviews, admin actions, and approvals are logged.
- Retention period: **[policy]** [e.g. 7 years] per applicable law.

6. Suspicious activity reporting

- **[policy]** A named **MLRO** reviews flagged activity and files STR/SAR with the relevant FIU as required. Internal flags (risk reviews, ops alerts) feed this process. MLRO: [name].

7. Account security controls (also AML-relevant)

- **Lockout:** user accounts lock after 5 failed PIN attempts for 30 minutes; staff logins lock similarly. Events are logged.
- **Staff governance:** role-based access control; sensitive actions (e.g. manual credits, admin sends, refunds, adding staff) require a **multi-admin approval** workflow; staff additions are department-scoped.
- Per-user transaction viewing and PDF/Excel report exports for oversight.

8. Governance & policy

- **[policy]** Board/management-approved AML/CFT policy, MLRO appointment, staff AML training, periodic independent review.
- Attach the signed AML/CFT policy document alongside this technical summary.

9. Honest control-maturity summary

Control	Status
Tiered KYC + enforced limits	Live
Transaction risk screening (velocity/caps/new-recipient)	Live
Reconciliation & security/fraud monitoring + alerts	Live
Account/staff lockouts + audit logging	Live
Staff RBAC + multi-approver workflow	Live

Government-ID verification provider	In progress
Sanctions/PEP screening	In progress / policy
MLRO, STR/SAR process, formal policy & training	Policy — confirm status

06 — Security & Data Protection

1. Data in transit

- All client ↔ server traffic over **HTTPS/TLS**; realtime over **WSS**.
- API hosts (Vercel/Railway) terminate TLS; HSTS in effect.

2. Data at rest

- PostgreSQL managed instance (Railway), provider-encrypted at rest.
- **User wallet secrets** encrypted with **AES-256-GCM** (per-user key derived from PIN + server secret; see Doc 04).
- **PINs / staff passwords** stored only as **bcrypt** hashes (cost 12).
- No card data is stored (no card acquiring).

3. Secrets management

Secret	Role
`WALLET_DERIVATION_SECRET`	Derives user wallets — highest sensitivity
`ENCRYPTION_KEY` (+ `_PREVIOUS`)	At-rest key encryption + rotation
`JWT_SECRET`	Session token signing
Stellar platform keys (gas/fee)	Platform wallet signing
Partner API keys, push keys, DB URL	Integrations

- Held as platform environment variables with restricted access.
- **Rotation:** at-rest encryption key supports dual-key rotation; partner/API keys rotated on a schedule and on suspected exposure.
- **Roadmap:** move wallet-derivation + signing into a **KMS/HSM-backed signing service** so application servers never handle raw user keys.

4. Authentication & access control

- Users: phone + PIN → JWT (short-lived) + refresh token.
- Staff/admin: separate identities, **role-based access control** (super-admin, department heads, department staff), least privilege by department.
- Sensitive operations require a **multi-approver** workflow.
- **Lockouts:** users (5 fails / 30 min) and staff logins; logged as security events.

5. Application security

- Server-side validation (schema validation) on all inputs.
- Rate limiting on sensitive endpoints and socket events.

- Pre-flight **fraud/risk screening** before value movement (Doc 05).
- Async errors are contained so a single failure cannot hang the service.

6. Infrastructure & monitoring

- Managed hosting (Vercel frontend, Railway backend + Postgres).
- **Error tracking:** Sentry (frontend + backend).
- **Ops monitor:** automated reconciliation, security, and fraud checks with throttled operator alerts (webhook + push).
- Audit logs for admin/staff actions and approvals.

7. Privacy & messaging

- Chat is designed around private messaging; message **previews are not shown on lock-screen notifications** (sender name + generic preview only).
- Personal data is collected for KYC/operation only; access is role-restricted.
- Data-subject handling and retention per applicable law: **[policy]**.

8. Incident response

- **[policy]** Documented IR plan: detection (Sentry + ops alerts) → triage → containment (freeze accounts, rotate secrets) → notification → post-mortem.
- Ability to **freeze** an account and to **block** transactions exists in the platform.
- Breach-notification obligations per applicable law: **[policy]**.

9. Business continuity

- Database backups: **[state cadence/retention]**.
- The deterministic wallet model means user wallets can be re-derived for recovery from the master secret + phone (Doc 04) — protect that secret accordingly.

10. Honest security-maturity summary

Area	Status
TLS everywhere, hashed PINs, AES-GCM key encryption	Live
RBAC + multi-approver + lockouts + audit logs	Live
Risk screening + reconciliation + ops alerting	Live
KMS/HSM-backed signer (key isolation)	Planned (top priority)
Independent pen-test / security audit	[planned / done]
Formal IR plan, backup policy, DPO/privacy policy	[confirm status]

07 — Yellow Card (On/Off-Ramp Partner) Integration

> How OlomiPay uses the partner's API. Confirms the division of responsibility: **the partner performs all local fiat (TZS / M-Pesa) collection, conversion, and payout under its licences**; OlomiPay orchestrates the user experience and holds value as USDC on Stellar.

1. Division of responsibility

Responsibility	Partner Olo(Yellow Card)
User app, UX, support	[Yes]
KYC of end users + limits	[Yes] (partner may also screen)
Holding USD value (USDC on Stellar)	[Yes] (custodial)
Local fiat collection (M-Pesa, TZS)	[Yes]
Fiat ↔ USDC conversion & FX	[Yes]
Local payout to mobile money	[Yes]
Licensing for the fiat leg	[Yes]

2. Operations used (partner API)

Flow	Partner capability used	Trigger
Channels / rates	List active channels, FX rates	App displays options
Fee/quote	Calculate fees & local payout	Before confirm
Deposit (collection)	Collect TZS via M-Pesa → USDC to OlomiPay/user	User adds money
Withdraw (payout)	USDC → TZS to user's M-Pesa	User withdraws
Cross-border payout	USDC → local currency in other markets	Remittance
Webhooks	Async status updates (settled/failed)	Partner → OlomiPay

3. Deposit sequence

```
1. User requests deposit of X TZS
2. OlomiPay → Partner: create deposit/collection order (internal ref = idempotency key)
3. Partner collects TZS from user via M-Pesa, converts to USDC
4. Partner → OlomiPay webhook: order completed
5. OlomiPay credits the user's USD (USDC) balance; notifies the user
```

4. Withdrawal / remittance sequence

```
1. User authorises withdrawal with PIN; OlomiPay runs tier-limit + AML checks
2. OlomiPay debits the user's USDC
3. OlomiPay → Partner: create payout order (USDC → local currency, recipient MSISDN)
4. Partner pays out to mobile money; → OlomiPay webhook with final status
5. OlomiPay marks the transaction settled/failed; notifies the user
```

5. Security of the integration

- Partner API credentials stored as restricted environment secrets (Doc 06).
- **Webhook authenticity** verified (signature/secret) before acting on a status.
- Every order carries an **idempotency reference** to prevent double-processing.
- Sandbox/testnet configuration supported for partner certification before go-live.

6. Settlement & reconciliation

- OlomiPay reconciles partner order outcomes against internal transaction records; mismatches are flagged to operators.
- Platform USDC vs. user liabilities reconciled automatically each cycle.

7. Information OlomiPay can share with the partner

- End-user KYC level and (on lawful request) identity details for a transaction.
- Transaction references, amounts, and status for dispute/AML cooperation.
- This technical pack + AML/KYC policy (Docs 04–06).

8. Open items to confirm with the partner

1. Written confirmation that OlomiPay's **custodial** use case is covered under the partner's licences for **Tanzania**.
2. Division of AML responsibility (who screens whom) documented.
3. Supported channels, limits, settlement timing, and webhook spec for go-live.
4. Production credentials + certification steps.

08 — Wallet Key Hardening Plan (KMS/HSM-backed signer)

> The single highest-leverage security item. Today, `WALLET_DERIVATION_SECRET` (and `ENCRYPTION_KEY`) are environment variables read by the application servers, and from the derivation secret + a phone number **every** user's private key can be reconstructed. Compromise of that one value = total loss of all user funds. This plan removes that single point of catastrophic failure.

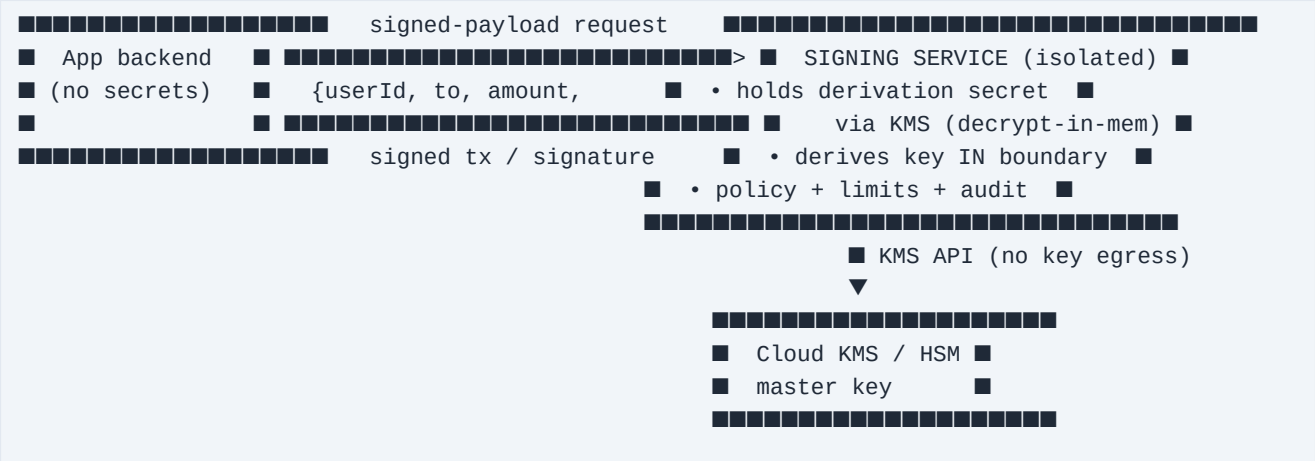
1. Threat being addressed

- **Master-secret exfiltration:** anyone who reads `WALLET_DERIVATION_SECRET` (leaked env, compromised host, malicious insider, log/backup exposure) can derive and drain every wallet.
- **Blast radius:** 100% of user funds, instantly, irreversibly.
- **Current mitigations:** env-var access control + wallet/fee separation — necessary but not sufficient for scale.

2. Objective (end state)

1. The derivation secret lives **only inside a hardened boundary** (cloud KMS/HSM or an isolated signing service), never in app memory or app env.
2. Application servers **never see raw user private keys** — they request "sign this specific transfer for user X" and receive a signed transaction.
3. Every signing request is **authenticated, authorised, rate-limited, and audit-logged**, with anomaly alerts.

3. Target architecture



- The signing service is a **separate deployment** with its own minimal network surface (no public ingress; only the backend can call it, over mTLS / signed internal auth).
- The master secret is stored in **KMS** (e.g. AWS KMS, GCP KMS, Cloudflare, or a dedicated HSM). The service decrypts it into memory at boot (or, better, performs derivation/signing via KMS so the raw key never leaves KMS).

4. Phased rollout (low-risk, incremental)

Phase 0 — Immediate mitigations (days) - Restrict who/what can read `WALLET_DERIVATION_SECRET` / `ENCRYPTION_KEY` to the absolute minimum; remove from any non-essential service/log. - Add an **alert** on access to / change of these secrets. - Confirm secrets are **not** in backups, error reports (Sentry scrubbing), or client bundles. - Document an incident runbook for suspected exposure (freeze + sweep).

Phase 1 — Extract an internal signing service (1–2 weeks) - Move all code paths that call `deriveKeypairFromPhone` / `getUserKeypair` behind a single internal service interface (`POST /sign`). - The backend stops importing the secret; it calls the signer over an authenticated internal channel (mTLS or signed service token). - Signer enforces **policy** (per-user/tier limits re-checked, allow-listed operations) and **audit-logs** every signature.

Phase 1 is implemented (code): the app now sources its sensitive secrets through a single provider (`services/secrets.ts`) with an env fallback, so the master secret can be moved out of plain platform env into a managed store **without any code change**. To activate **Infisical** (free tier): 1. Create an Infisical project; in its `prod` environment add `WALLET_DERIVATION_SECRET`, `ENCRYPTION_KEY`, `JWT_SECRET`, `JWT_REFRESH_SECRET` (and any others you want vaulted). 2. Create a **Machine Identity (Universal Auth)** with read access to the project. 3. Set these env vars on the backend (Railway): `INFISICAL_CLIENT_ID`, `INFISICAL_CLIENT_SECRET`, `INFISICAL_PROJECT_ID`, `INFISICAL_ENV=prod` (and `INFISICAL_API_URL` only if self-hosting). 4. Deploy; confirm the boot log shows `[secrets] loaded N secret(s) from managed store (prod)`. 5. **Then remove** the migrated secrets from plain Railway env — the app now reads them from Infisical, with access logged there.

> [!] Do **not** remove `WALLET_DERIVATION_SECRET` / `ENCRYPTION_KEY` from env until you've confirmed Infisical is serving them (the boot log + a working login/transfer), or wallets won't derive.

Phase 2 — KMS/HSM-backed key custody (2–4 weeks) - Store the master secret in **KMS**; the signer obtains it only via KMS decrypt at boot, or derives/signs through KMS so the raw key never egresses. - Lock down the signer host: no shell, minimal image, network egress allow-list, read-only FS. - Add **rate limits + anomaly detection** on signing volume/patterns.

Phase 3 — Optional advanced custody (later) - Evaluate **threshold/MPC signing** (no single machine ever holds a full key) or a managed qualified-custody provider for the highest-value treasury. - Re-assess whether a partial **non-custodial** option is desirable for advanced users (trade-off: loses phone-only recovery).

5. The rotation problem (important)

Because addresses are derived from the master secret, **rotating it changes every user's address** — you cannot rotate in place without migrating funds. Therefore: - Treat the secret as **long-lived and protected by access control**, not by frequent rotation. - If rotation is ever required (suspected compromise), the procedure is a **sweep migration**: derive old keys inside the boundary, generate new wallets under a new secret, and move balances on-chain in a controlled batch. - Keep this sweep runbook written and tested (table-top) in advance.

6. Controls checklist (target)

Control	Target
Master secret in KMS/HSM, not app env	[Yes]
App servers never hold raw user keys	[Yes]
Signing isolated behind authenticated internal API	[Yes]
Per-request policy + tier-limit re-check	[Yes]
Full signing audit log + anomaly alerts	[Yes]
Secret access alerting; excluded from logs/backups/bundles	[Yes]
Documented + tested key-compromise sweep runbook	[Yes]
Independent security review / pen-test of the signer	[Yes]

7. Why this matters for partner/regulator review

A reviewer **will** notice "one master key controls all funds." Presenting this plan — with Phase 0 already done — turns a red flag into evidence of mature custody-risk management, and is the right thing to do regardless of optics.

09 — Key-Compromise Response Runbook

> What to do if `WALLET_DERIVATION_SECRET` or `ENCRYPTION_KEY` is suspected exposed. Because every user wallet derives from the master secret, exposure is the **highest-severity** incident. Print this; don't improvise it under stress.

Severity & owners

- **Severity: CRITICAL (P0).** Potential total loss of user funds.
- **Incident commander:** [name / role]
- **Backup:** [name]
- **Notify immediately:** CTO, CEO, Compliance/MLRO, on/off-ramp partner contact.

Triggers (any of)

- Master secret seen in a log, error report, screenshot, chat, commit, or backup.
- A host with access to the secret is compromised or behaves anomalously.
- Unexplained outbound transfers / balance drops across multiple users.
- A departing insider had access and access wasn't revoked.

Step 1 — CONTAIN (minutes)

1. **Freeze movement.** Put the platform into a state where **no outbound transfers are processed** (disable send/withdraw/remit/agent-payout — e.g. maintenance flag / pause the signing path).
2. **Rotate everything reachable now:** partner API keys, `JWT_SECRET`, DB credentials, admin/staff sessions (force re-login). This limits lateral use.
3. **Revoke access** to the secret: rotate cloud credentials, pull the secret from any host that doesn't strictly need it, disable suspected accounts.
4. **Preserve evidence:** snapshot logs, access logs, and the AdminAuditLog (incl. KYC-access trail) before anything is wiped.

Step 2 — ASSESS (first hour)

- Confirm whether the secret was actually exposed vs. suspected.
- Check on-chain: pull recent transfers for platform + user wallets; identify any unauthorised movements (Horizon).
- Quantify exposure window (when could the secret have leaked → now).

Step 3 — SWEEP MIGRATION (if exposure confirmed)

Because addresses derive from the secret, you cannot "rotate in place" — you **migrate funds to wallets under a NEW secret**: 1. Generate a **new** `WALLET_DERIVATION_SECRET` inside the secure boundary (KMS/HSM if available — see Doc 08). 2. For each user, derive the **old** key (old secret + phone) **inside the boundary** and the **new** address (new secret + phone). 3. **Sweep** each user's USDC (and dust XLM) from old → new address on-chain, in controlled batches, sponsored by the gas wallet. Log every move. 4. Update each user's stored `stellarPubKey` (+ re-encrypt secret under new scheme) to the new address. 5.

Reconcile: sum of swept balances == pre-sweep liabilities. Investigate any gap. 6. Keep the old secret **only** until the sweep is verified complete, then destroy.

> Pre-write and table-top test this sweep script **before** you ever need it. Doing it for the first time during a live incident is how funds get lost.

Step 4 — RESTORE

- Re-enable transfers only after: new secret in place, sweep verified, reconciliation balanced, and the exposure vector closed.
- Force a fresh app session for all users; confirm balances display correctly.

Step 5 — NOTIFY & REPORT

- **Users:** clear, honest notice if funds/PII were at risk (per law).
- **Partner (Yellow Card):** per your agreement's incident clause.
- **Regulator / FIU:** breach notification per applicable Tanzanian law and AML/CFT obligations (MLRO leads). Timelines: [confirm with counsel].
- **Data protection:** if PII (KYC docs) exposed, follow breach-notification law.

Step 6 — POST-INCIDENT

- Root-cause analysis; close the vector (e.g. move secret to KMS — Doc 08).
- Update this runbook with lessons learned.
- Independent review of the response.

Standing prevention checklist (do these now — Phase 0)

- [ ] Master secrets excluded from logs, Sentry (scrubbing in place), backups, bundles.
- [ ] Alert on access to / change of the master secrets.
- [ ] Least-privilege access; list exactly who/what can read them.
- [ ] Sweep-migration script written + table-top tested.
- [ ] This runbook printed and owners assigned.
- [ ] Roadmap: move signing into a KMS/HSM-backed signer (Doc 08).